



**Mühendislik ve Doğa Bilimleri Fakültesi
Bilgisayar Mühendisliği**

**BM-302 Bilgisayar Ağları Dönem Projesi
Teknik Raporu**

NetProbe: UDP Üzerinde Güvenilir Dosya Transferi

Akif Adnan – 20360859106

Haziran 2026

İÇİNDEKİLER

1. Giriş.....	4
2. Problem Tanımı.....	4
3. Sistem Mimarisi.....	5
3.1 Bileşen Yapısı.....	5
3.2 Veri Akışı	5
4. Protokol Tasarımı	6
4.1 Paket Formatı	6
4.2 ARQ Protokolleri	6
4.2.1 Stop-and-Wait (Pencere = 1).....	6
4.2.2 Go-Back-N Kayan Pencere ($W > 1$).....	6
4.3 Bütünlük Katmanı	6
4.3.1 CRC-32 — Paket Düzeyi.....	6
4.3.2 MD5 — Uçtan Uca Bütünlük.....	6
5. Gerçekleme Detayları.....	7
5.1 Kullanılan Teknolojiler.....	7
5.2 Çok İş Parçacıklı Mimari	7
5.3 Hata İşleme	8
5.4 Bonus Özellikler.....	8
6. Deney Ortamı.....	8
6.1 Donanım ve Yazılım Altyapısı	8
6.2 Kontrol Değişkenleri.....	8
6.3 Ölçüm Yöntemi	9
7. Performans Metrikleri	9
7.1 Tanımlar ve Hesaplama Yöntemleri.....	9
Goodput (Faydalı Verimlilik)	9
Throughput (Toplam Verim)	9
Yeniden İletim Oranı	9
Kanal Kullanım Oranı	9
Bant Genişliği-Gecikme Çarpımı (BDP).....	9
7.2 Metrik Toplama Altyapısı	10
8. Sonuçlar ve Tartışma	10
8.1 Deney 1 — Pencere Boyutunun Goodput'a Etkisi	10
8.2 Deney 2 — Paket Kaybının Goodput'a Etkisi	10
8.3 Stop-and-Wait ile Go-Back-N Karşılaştırması	11
9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları.....	11
9.1 Yarış Koşulu — Paylaşılan base Değişkeni	11
9.2 CRC Hesabında Negatif Tamsayı	12
9.3 SSE Akışının Flask Önbelleği Tarafından Tamponlanması	12

9.4 CSV Günlük Ayırıştırma Hatası.....	12
9.5 Port 5000'de Çakışan Süreç	12
9.6 GBN Yeniden İletim Döngüsünde Sayaç Tutarsızlığı	12
10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler	12
10.1 Sonuç	12
10.2 Gelecekte Yapılabilecek Geliştirmeler.....	13
Kaynakça	13

1. Giriş

Bu rapor, Bursa Teknik Üniversitesi BM-302 Bilgisayar Ağları dersi kapsamında geliştirilen NetProbe projesini belgelemektedir. NetProbe; UDP (User Datagram Protocol) soket programlama, ARQ (Automatic Repeat reQuest) güvenilirlik mekanizmaları ve ağ performans metriklerini uygulamalı olarak incelemeye yönelik, sıfırdan tasarlanmış bir dosya transfer sistemidir.

İnternet trafiğinin büyük bölümü TCP üzerinden taşınmakta; ancak gerçek zamanlı uygulamalar (video konferans, çevrimiçi oyun, VoIP) gecikme hassasiyetleri nedeniyle UDP'yi tercih etmektedir. TCP'nin güvenilirlik katmanını şeffaf biçimde inceleyebilmek için, aynı işlevselliğin UDP üzerine elle kodlanması bu proje için en uygun öğrenme ortamını sağlamaktadır.

Proje üç temel hedefi bir arada gerçeklemektedir: (i) UDP datagram soketleri üzerinden güvenilir veri iletimi, (ii) Stop-and-Wait ve Go-Back-N ARQ protokollerinin karşılaştırmalı analizi ve (iii) Flask tabanlı gerçek zamanlı web arayüzü aracılığıyla ağ performans metriklerinin görselleştirilmesi.

2. Problem Tanımı

UDP, IP üzerinde çalışan bağlantısız bir taşıma katmanı protokolüdür. Sekiz baytlık minimal başlık yapısı, bağlantı kurma yükü olmaması ve gönderici tarafında bekleme gerektirmemesi UDP'ye düşük gecikme avantajı sağlar; ancak aynı nedenlerle aşağıdaki güvenilirlik garantileri bulunmamaktadır:

- Teslimat garantisi yok: Gönderilen bir datagram, ağ üzerinde sessizce düşürülebilir.
- Sıra garantisi yok: Ardışık gönderilen iki datagram, alıcıya farklı sırada ulaşabilir.
- Bütünlük garantisi yok: Bit hatası oluşmuş bir datagram, UDP tarafından reddedilmez.
- Tıkanıklık kontrolü yok: Gönderici, ağı aşırı yükleyecek hızda veri gönderebilir.

Bu proje, söz konusu eksiklikleri uygulama katmanında elle yazılmış mekanizmalarla gidermektedir. Çözülmesi hedeflenen alt problemler şunlardır:

Alt Problem	Çözüm Yaklaşımı
Paket kaybının tespiti ve giderimi	Sıra numaraları + ACK mekanizması + zaman aşımı yeniden iletimi
Sıra dışı paket tespiti	Sekans numarası ile beklenen paket karşılaştırması
Bit bozulmalarının tespiti	Her pakette CRC-32 sağlama toplamı
Uçtan uca bütünlük doğrulaması	Transfer sonrası MD5 parmak izi karşılaştırması
Düşük verimlilik (Stop-and-Wait)	Kayan pencere (Go-Back-N) ile boru hattı iletimi

Ayrıca, localhost üzerinde gerçek ağ koşullarını simüle edebilmek amacıyla yazılımsal paket kaybı mekanizması da uygulanmıştır. Bu sayede protokol davranışı, kontrollü kayıp oranları (%0–%20) altında ölçülebilmektedir.

3. Sistem Mimarisi

3.1 Bileşen Yapısı

NetProbe üç ana bileşenden oluşmaktadır: Gönderici (sender), Alıcı (receiver) ve Web İzleme Arayüzü (dashboard). Tüm bileşenler Python 3.11 ile geliştirilmiş ve yalnızca standart kütüphane ile Flask çerçevesine bağımlıdır.

Modül	Dosya	Sorumluluk
Protokol sabitleri	protocol.py	Paket formatı, bayrak tanımları, sabitler
Gönderici	sender.py	ARQ döngüsü, kayan pencere, istatistik toplama
Alıcı	receiver.py	Paket doğrulama, yeniden sıralama, dosya yazma
Web arayüzü	app.py	Flask SSE API, CSV günlük okuma, deney yönetimi
Yardımcılar	utils.py	XOR şifreleme, zlib sıkıştırma, pcap yazıcı
Komut satırı	cli.py	Argüman ayrıştırma, uygulama başlangıç noktası

3.2 Veri Akışı

Tek bir dosya transferinin veri akışı aşağıdaki sırayla gerçekleşir:

- Kullanıcı, web arayüzünden dosya yolunu, pencere boyutunu ve kayıp oranını girer.
- Flask, arka planda bir sender thread'i başlatır ve bu transfere özel bir olay kuyruğu (queue.Queue) oluşturur.
- Tarayıcı, EventSource API üzerinden SSE akışını dinlemeye başlar.
- Sender, dosyayı 4 KB'lık parçalara böler, ARQ döngüsünü başlatır ve her olayı kuyruğa yazar.
- Flask, kuyruktan okunan olayları SSE formatında tarayıcıya iletir.
- Receiver, gelen paketleri doğrular, onaylar, birleştirir ve dosyayı diske yazar.
- Transfer tamamlandığında MD5 karşılaştırması yapılır ve sonuç arayüze iletilir.

4. Protokol Tasarımı

4.1 Paket Formatı

Her UDP datagramının yükü, 19 baytlık özel bir ikili başlık ile başlar. Başlık, Python struct modülü ve büyük-endian ağ byte sıralaması (! öneki) kullanılarak serileştirilmektedir.

Ofset (B)	Boyut (B)	Alan	Açıklama
0	4	MAGIC	0xDEAD — protokol tanımlayıcı, yabancı paketleri eler
4	1	FLAGS	DATA=0x01, ACK=0x02, FIN=0x04, ERR=0x08 (bit maskeleme)
5	4	SEQ_NUM	Gönderici sıra numarası
9	4	ACK_NUM	Onaylanan son sıra numarası (uint32)
13	2	DATA_LEN	Yük boyutu, bayt cinsinden (uint16)
15	4	CRC32	Başlık + veri üzerinden zlib.crc32 sağlama toplamı
19	N	DATA	Asıl yük verisi

4.2 ARQ Protokolleri

4.2.1 Stop-and-Wait (Pencere = 1)

Gönderici bir paket gönderir ve ACK alana kadar bir sonrakini göndermez. Son derece basit ancak verimsizdir.

4.2.2 Go-Back-N Kayan Pencere ($W > 1$)

Gönderici, ACK beklemeksizin W adet paketi art arda gönderebilir. Pencere sınırına ulaşıldığında ilk gönderilen paketin ACK'i beklenir. Bir paket kaybolduğunda, o paketin sıra numarasından itibaren pencere içindeki tüm paketler yeniden iletilir.

4.3 Bütünlük Katmanı

4.3.1 CRC-32 — Paket Düzeyi

CRC-32, veriyi büyük bir ikili polinom olarak ele alır ve sabit bir üreteç polinomu ile böler; kalan 32 bit CRC değeridir. Tek bit hatalarını ve 32 bit uzunluğuna kadar tüm burst hatalarını garantili olarak tespit eder.

4.3.2 MD5 — Uçtan Uca Bütünlük

Transfer tamamlandıktan sonra orijinal dosyanın MD5 özeti ile alıcıda yeniden birleştirilmiş verinin MD5 özeti karşılaştırılır. Eşleşme sağlanamadığında 'Bütünlük Hatası' rapor edilir. CRC-32 ile MD5 birbirini tamamlar: CRC → tek paket bit hatası; MD5 → tüm dosya uçtan uca doğruluk.

5. Gerçekleme Detayları

5.1 Kullanılan Teknolojiler

Kütüphane	Versiyon	Kullanım Amacı	Kaynak
socket	stdlib	UDP soket oluşturma ve datagram I/O	Python Standart Kütüphane
struct	stdlib	Binary başlık serileştirme (big-endian)	Python Standart Kütüphane
zlib	stdlib	CRC-32 sağlama toplamı + veri sıkıştırma	Python Standart Kütüphane
hashlib	stdlib	MD5 uçtan uca özeti, SHA-256 anahtar akışı	Python Standart Kütüphane
threading	stdlib	Çok iş parçacıklı kayan pencere yönetimi	Python Standart Kütüphane
queue	stdlib	Thread-safe olay iletimi (SSE için)	Python Standart Kütüphane
random	stdlib	Yazılımsal paket kaybı simülasyonu	Python Standart Kütüphane
Flask	3.x	Web sunucusu, REST API, SSE akışı	PyPI — MIT Lisansı
Chart.js	4.4.0	Tarayıcı tabanlı canlı grafikler	CDN — MIT Lisansı

Not: Hiçbir transfer kütüphanesi yok.

5.2 Çok İş Parçacıklı Mimari

Kayan pencere mekanizması doğası gereği iki eş zamanlı görevi gerektirir: paket gönderme + ACK alma. Tek iş parçacığıyla `recvfrom()` çağrısı blokladığından zaman aşımı kontrolü ve paket gönderimi durma noktasına gelir. Bu nedenle iki iş parçacığı kullanılmaktadır:

- Ana iş parçacığı: Pencereyi dolduran paketleri gönderir ve zaman aşımı kontrolü yaparak GBN yeniden iletimine karar verir.
- ACK alıcı iş parçacığı (daemon): Soket üzerinde sürekli dinleme yapar; ACK geldiğinde `threading.Lock` ile korunan base değişkenini güncelleyerek pencereyi ilerletir.

Paylaşılan base değişkeni iki iş parçacığı tarafından eş zamanlı erişime açık olduğundan mutex kilidi (`threading.Lock`) zorunludur. Kilitsiz erişimde yarış koşulu (race condition) oluşur: iki iş parçacığı aynı anda okuma-yazma yaptığında base tutarsız bir değer alabilir ve protokol bütünlüğü bozulur.

5.3 Hata İşleme

Hata Durumu	Tespit Yöntemi	Uygulanan Çözüm
Paket kaybı	ACK zaman aşımı	GBN: base'den itibaren yeniden iletim
Bit bozulması	CRC-32 uyumsuzluğu	Paket sessizce atılır, zaman aşımı tetiklenir
Sıra dışı paket	seq != expected_seq	GBN'de paket atılır, ACK tekrar gönderilmez
Çift (yinelenen) paket	seq in received_set	set ile O(1) tespitte atılır, ACK yeniden gönderilir
Yabancı datagram	magic != 0xDEAD	Paket işlenmeden atılır
Ağ zaman aşımı	socket.settimeout(RTO)	Exception yakalanır, yeniden deneme sayacı artırılır

5.4 Bonus Özellikler

Özellik	Teknik Uygulama
XOR akış şifrelemesi	SHA-256 tabanlı 32 baytlık anahtar akışı, döngüsel XOR
zlib veri sıkıştırma	Gönderimden önce zlib.compress(); alımda zlib.decompress()
.pcap çıktısı (Wireshark)	libpcap ikili formatı struct ile elle oluşturuldu, link type 228
Çoklu istemci desteği	sessions[(host,port)] sözlüğü ile bağımsız oturum yönetimi
Bant genişliği sınırlama	Paket arası dinamik time.sleep() ile hedef bps kontrolü
CSV olay günlüğü	csv.writer ile zaman damgalı transfer olayları kaydı
Flask SSE arayüzü	queue.Queue + text/event-stream, Chart.js canlı grafikler
Büyük dosya desteği	Otomatik parçalama ve alıcıda yeniden birleştirme

6. Deney Ortamı

6.1 Donanım ve Yazılım Altyapısı

Bileşen	Değer
İşletim sistemi	Windows 11 (loopback: 127.0.0.1)
Python sürümü	3.11.9
Ağ arayüzü	Loopback - fiziksel NIC kullanılmadı
Taşıma katmanı	UDP
Test dosya boyutu	10 MB (rastgele ikili veri, tekrarlanabilir seed)
Paket boyutu	4096 bayt (veri yükü, başlık hariç)
Zaman aşımı (RTO)	500 ms (tüm deneylerde sabit)
Web arayüzü portu	TCP 5000 (Flask geliştirme sunucusu)

6.2 Kontrol Değişkenleri

Deneyler iki bağımsız değişken üzerinde yürütülmüştür. Her ölçüm üç kez tekrarlanmış; ortalama değer raporlanmıştır.

- Deney 1 — Pencere Boyutu Etkisi: Pencere boyutu {1, 4, 8, 16, 32} olarak değiştirilmiş; paket kaybı %0 sabit tutulmuştur.
- Deney 2 — Paket Kaybı Etkisi: Kayıp oranı {%0, %5, %10, %20} olarak değiştirilmiş; pencere boyutu W=16 sabit tutulmuştur.

6.3 Ölçüm Yöntemi

Goodput değerleri Python'da `time.time()` ile milisaniye hassasiyetinde ölçülmüştür. Retransmission sayısı, GBN yeniden iletim döngüsünde artırılan bir sayaçtan okunmuştur. Tüm ölçüm verileri `transfer_log.csv` dosyasına yazılmış ve web arayüzünde gerçek zamanlı gösterilmiştir.

Not: *Localhost loopback arayüzü gerçek ağ gecikmesi üretmez. Bu nedenle paket kaybı yazılımsal olarak (`random.random() < loss_rate`) simüle edilmiştir. Gerçek WAN deneyleri için `tc-netem` veya benzer bir ağ emülatörü gereklidir.*

7. Performans Metrikleri

7.1 Tanımlar ve Hesaplama Yöntemleri

Goodput (Faydalı Verimlilik)

Alıcıya ulaşan faydalı veri miktarının geçen süreye oranıdır. Yeniden iletilen paketler sayılmaz.

$$\text{Goodput} = \text{Dosya_Boyutu (bayt)} / \text{Transfer_Suresi (saniye)}$$

Throughput (Toplam Verim)

Gönderici tarafından ağa verilen toplam baytın (yeniden iletimler dahil) süreye oranıdır.

$$\text{Throughput} = \text{Toplam_Gonderilen_Bayt} / \text{Transfer_Suresi}$$

Yeniden İletim Oranı

$$\text{Yeniden_Iletim_Orani} = \text{Yeniden_Iletilen_Paket} / \text{Toplam_Gonderilen_Paket}$$

Kanal Kullanım Oranı

Stop-and-Wait için teorik üst sınır:

$$U_{\text{StopWait}} = T_{\text{trans}} / (T_{\text{trans}} + \text{RTT})$$

Go-Back-N için W pencere boyutuyla:

$$U_{\text{GBN}} = \min(1, W * T_{\text{trans}} / (T_{\text{trans}} + \text{RTT}))$$

Bant Genişliği-Gecikme Çarpımı (BDP)

Kanalı tam kullanmak için eş zamanlı havada olması gereken veri miktarı:

$$\text{BDP} = \text{Bant_Genisligi (bps)} \times \text{RTT (saniye)}$$

Optimum pencere boyutu (paket cinsinden): $W_{\text{opt}} = \text{BDP} / \text{Paket_Boyutu}$

7.2 Metrik Toplama Altyapısı

Gönderici, her ACK alındığında ve her zaman aşımında aşağıdaki alanları içeren bir JSON olayını olay kuyruğuna (queue.Queue) ekler:

```
# Metrik olayı — sender.py
event = {
    'timestamp': time.time(),
    'seq': seq_num,
    'goodput_kbps': (bytes_acked / elapsed) / 1024,
    'throughput_kbps': (bytes_sent / elapsed) / 1024,
    'loss_rate': lost_count / sent_count,
    'retransmits': retransmit_count,
    'window_fill': (next_seq - base) / WINDOW_SIZE,
    'event_type': 'ACK' | 'RETRANSMIT' | 'TIMEOUT' | 'COMPLETE'
}
transfer_queue.put(event)
```

Flask, bu kuyruğu SSE akışına dönüştürerek tarayıcıya iletir. Chart.js her yeni olayda grafikleri günceller.

8. Sonuçlar ve Tartışma

8.1 Deney 1 — Pencere Boyutunun Goodput'a Etkisi

Paket kaybı %0 sabit tutularak pencere boyutu değiştirilmiş ve goodput ölçülmüştür:

Pencere Boyutu (W)	Goodput (KB/s)	Kanal Kullanımı	Yeniden İletim
1 (Stop-and-Wait)	~820	Dusuk (~%1)	0
4	~2.400	Orta (~%5)	0
8	~3.900	İyi (~%10)	0
16	~4.800	Yüksek (~%20)	0
32	~5.100	Doygunluk	0

[Pencere boyutuna göre goodput değişimi]

Tartışma: W=1'den W=16'ya geçişte goodput yaklaşık 6 kat artmıştır. W=32'de artış yavaşlamaktadır; bu durum, dar boğazın artık bağlantı değil işletim sistemi UDP socket tamponu ve Python yorumlayıcı yükü olduğunu göstermektedir. Teorik formül $W \times T_{\text{trans}} / (T_{\text{trans}} + \text{RTT})$ ile ölçülen değerler niteliksel olarak örtüşmektedir: pencere boyutu bant genişliği-gecikme çarpımını aştığında ek kazanım ortadan kalkar.

8.2 Deney 2 — Paket Kaybının Goodput'a Etkisi

Pencere boyutu W=16 sabit tutularak paket kaybı oranı değiştirilmiştir:

Kayıp Oranı	Goodput (KB/s)	Yeniden İletim Oranı	Bütünlük
%0	~4.800	%0	OK (MD5 eslemesi)
%5	~3.200	~%8	OK (MD5 eslemesi)
%10	~2.100	~%18	OK (MD5 eslemesi)
%20	~900	~%42	OK (MD5 eslemesi)

[Paket kaybı oranına göre goodput ve yeniden iletim oranı]

Tartışma: Goodput, kayıp oranıyla doğrusal değil, yaklaşık katlanmalı biçimde düşmektedir. Bunun nedeni GBN'nin yeniden iletim davranışdır: W=16 penceresiyle bir kayıp olayı, en fazla 16 paketin yeniden iletilmesini tetikler. Yüksek kayıp oranlarında bu yeniden iletilen paketlerin bir kısmı da kaybolabilir ve ikincil yeniden iletim dalgaları oluşur. %20 kayıp oranında ölçülen ~%42 yeniden iletim oranı bu çoğaltma etkisini somut biçimde ortaya koymaktadır.

Kritik bulgu: Tüm deney senaryolarında transfer tamamlandıktan sonra MD5 parmak izi karşılaştırması başarılı sonuç vermiştir. ARQ mekanizması, %20 kayıp oranında bile tek bir bitlik veri bozulmasına izin vermemiştir; bu durum protokolün güvenilirlik garantisini deneysel olarak doğrulamaktadır.

8.3 Stop-and-Wait ile Go-Back-N Karşılaştırması

Kriter	Stop-and-Wait	Go-Back-N (W=16)
Goodput (0% kayıp)	~820 KB/s	~4.800 KB/s (+485%)
Goodput (10% kayıp)	~750 KB/s	~2.100 KB/s
Uygulama karmaşıklığı	Cok basit	Orta (iki thread, lock)
Alici tamponu gereksinimi	Yok	Yok (GBN)
Yeniden iletim verimliliği	Optimal	Dusuk (kaskad)

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

9.1 Yarış Koşulu — Paylaşılan base Değişkeni

Sorun: İlk uygulamada base değişkenine iki iş parçacığı eş zamanlı eriştiğinde, ana iş parçacığı eski base değeriyle pencereyi hesaplarken ACK iş parçacığı base'i güncelliyordu. Bu durum nondeterministik paket çiftlenmesine ve sıra numarası tutarsızlığına yol açtı.

Çözüm: threading.Lock() ile mutex koruması eklendi. base üzerindeki tüm okuma ve yazma işlemleri with lock: bloğuna alındı. Kilitlenme (deadlock) riskini önlemek için lock, herhangi bir bloklanabilir çağrı (recvfrom, sleep) dışında tutuldu.

9.2 CRC Hesabında Negatif Tamsayı

Sorun: Python'da `zlib.crc32()` bazı platformlarda negatif değer döndürmektedir; bu değer `struct.pack('l', crc)` çağrısında `struct.error: ubyte format requires 0 <= number <= 4294967295` hatasına neden oluyordu.

Çözüm: `crc = zlib.crc32(data) & 0xFFFFFFFF` maskesi uygulandı. Python'un keyfi hassasiyetli tamsayıları 32 bitlik işaretli aralığa zorlanarak tutarlı davranış sağlandı.

9.3 SSE Akışının Flask Ön Belleği Tarafından Tamponlanması

Sorun: Flask'ın geliştirme sunucusu, SSE yanıtlarını varsayılan olarak tamponluyor; bu nedenle metrikler anlık değil toplu biçimde tarayıcıya ulaşıyordu. Canlı grafik güncelleme işlevselliği bozuluyordu.

Çözüm: SSE yanıt başlıklarına `Cache-Control: no-cache` ve `X-Accel-Buffering: no` eklendi. `Response` nesnesi `mimetype='text/event-stream'` ile oluşturuldu ve generator fonksiyonu her olaydan sonra `sys.stdout.flush()` çağırıldı.

9.4 CSV Günlük Ayırıştırma Hatası

Sorun: Olay günlüğü detay alanları virgül içerdiğinde `line.split(',', 3)` yöntemi alanları yanlış ayırıyordu; arayüzde detay metinleri bozuk görünüyordu.

Çözüm: Ayırıştırma `next(csv.reader([line]))` ile değiştirildi. `csv.reader`, tırnak içine alınmış (quoted) alanlardaki virgülleri doğru biçimde işlemektedir.

9.5 Port 5000'de Çakışan Süreç

Sorun: Geliştirme sürecinde Flask sunucusu çöktükten sonra yeniden başlatıldığında `Port 5000 is already in use` hatası alındı. İşletim sistemi `TIME_WAIT` durumundaki soketi henüz serbest bırakmamıştı.

Çözüm: `app.run()` çağrısına `use_reloader=False` parametresi eklendi ve soket seçeneği `SO_REUSEADDR` etkinleştirildi. Geliştirme ortamında süreç yönetimi için `stop-process` ile çakışan Python süreçleri sonlandırıldı.

9.6 GBN Yeniden İletim Döngüsünde Sayaç Tutarsızlığı

Sorun: Zaman aşımı tetiklendiğinde `next_seq = base` atanarak pencere sıfırlanıyor, ancak bu süreçte hangi paketlerin yeniden iletilişi doğru sayılmıyordu. Retransmission oranı metriği hatalı hesaplanıyordu.

Çözüm: Yeniden iletim sayacına `next_seq - base` eklenerek `retransmit_count` doğru güncellendi. Sayaç artırımını her zaman aşımı olayında `next_seq = base` atamasından önce gerçekleştirildi.

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

10.1 Sonuç

NetProbe projesi, UDP protokolü üzerine güvenilir dosya transferi gerçekleştiren, ARQ mekanizmalarını uygulamalı olarak karşılaştıran ve performans metriklerini gerçek zamanlı görselleştiren eksiksiz bir sistem ortaya koymaktadır.

Deneysel bulgular teorik beklentilerle örtüşmektedir: Go-Back-N kayan pencere, Stop-and-Wait'e kıyasla goodput'u $W=16$ için ~6 kat artırmıştır. Paket kaybı goodput'u orantısız biçimde düşürmekte; bu durum GBN'nin kaskad yeniden iletim davranışını somutlaştırmaktadır. %20 kayıp oranında bile MD5 bütünlük doğrulaması başarılı olmuş; ARQ mekanizmasının güvenilirlik garantisi kanıtlanmıştır.

Proje, ağ protokolü tasarımının üç temel ilkesini uygulamalı biçimde pekiştirmiştir: güvenilirlik mekanizmalarının karmaşıklığı, performans-doğruluk dengesi ve eşzamanlı sistemlerde senkronizasyonun kritikliği.

10.2 Gelecekte Yapılabilecek Geliştirmeler

- Seçici Yineleme (SR) ARQ: Yalnızca eksik paketin yeniden iletilmesi ile GBN'nin kaskad yeniden iletim maliyeti ortadan kaldırılabilir; yüksek kayıp oranlarında belirgin goodput iyileştirmesi beklenmektedir.
- Dinamik Pencere Boyutu (Congestion Control): RTT ölçümüne göre pencere boyutunun dinamik ayarlanması ile TCP'nin tıkanıklık kontrolüne benzer bir davranış elde edilebilir.
- DTLS Şifreleme: XOR akış şifresi yerine DTLS (Datagram TLS) entegrasyonu ile üretim düzeyinde kimlik doğrulama ve gizlilik sağlanabilir.
- Gerçek WAN Deneyleri: tc-netem veya Mininet ile gerçekçi gecikme ve kayıp profillerinin simülasyonu; BDP hesabının gerçek ağ koşullarında doğrulanması.
- QUIC Protokolü Karşılaştırması: Benzer tasarım kararları alan QUIC ile NetProbe'un karşılaştırmalı analizi; özellikle head-of-line blocking farkının ölçülmesi.
- Adaptif Zaman Aşımı (TCP Jacobson Algoritması): Sabit RTO yerine ölçülen RTT örneklerinden dinamik RTO hesaplama ile yeniden iletim verimliliğinin artırılması.

Kaynakça

- [1] Kurose, J. F., & Ross, K. W. (2021). Computer Networking: A Top-Down Approach (8. baskı). Pearson Education.
- [2] Tanenbaum, A. S., & Wetherall, D. (2011). Computer Networks (5. baskı). Prentice Hall.
- [3] Postel, J. (1980). RFC 768 — User Datagram Protocol. IETF.
- [4] Defense ARPA. (1981). RFC 793 — Transmission Control Protocol. IETF.
- [5] Python Software Foundation. (2024). socket — Düşük düzey ağ arayüzü. <https://docs.python.org/3/library/socket.html>
- [6] Python Software Foundation. (2024). struct — C yapılarının ikili veriye paketlenmesi. <https://docs.python.org/3/library/struct.html>
- [7] Pallets Projects. (2024). Flask Belgesi. <https://flask.palletsprojects.com/>
- [8] Chart.js Katkıda Bulunanlar. (2024). Chart.js Belgesi. <https://www.chartjs.org/docs/>